LING 570 HW2 Readme

Vinisha Bala Dhayanidhi

October 11, 2026

Q3: Right-regular grammar to FSA

Given a right-regular grammar $G = (N, \Sigma, P, S)$, where N is the set of non-terminals, Σ the terminals, P the rules and S the start symbol, I built the FSA $M = (\Sigma_M, Q, I, F, \delta)$ as follows.

- **Alphabet:** $\Sigma_M = \Sigma$, the terminals of G.
- **States:** one state for every non-terminal, plus one new state $q_f \notin N$. So $Q = N \cup \{q_f\}$. In my code q_f is named `qf`; if `qf` is already a non-terminal, my code gives the new state a different name so the two states don't clash.
- **Initial state:** $I = S$, the left-hand side of the first rule in the input file.
- **Final states:** $F = \{q_f\}$. This allows only one final state.
- **Transitions:** one arc for each rule in P:

Rule in P		Arc in δ
$A \to t\,B$	(t a terminal)	$A \xrightarrow{t} B$
$A \to t$	(t a terminal)	$A \xrightarrow{t} q_f$
$A \to \varepsilon$	(`*e*` in the file)	$A \xrightarrow{\varepsilon} q_f$

(If a rule `A *e* B` ever appeared, it would give an ε-arc $A \to B$.)

Why this is correct. A derivation

$$S \Rightarrow t_1 A_1 \Rightarrow t_1 t_2 A_2 \Rightarrow \cdots \Rightarrow t_1 t_2 \cdots t_n$$

corresponds to a path from S to q_f that reads $t_1 t_2 \cdots t_n$, and every path from S to q_f gives back such a derivation. So M accepts exactly $L(G)$. The FSA can be an NFA (it can have ε-arcs and also many arcs on one symbol).

The first arc written to the output comes from the first rule of the grammar, so the start state is the first state on the 2nd line and the first line is the final state. For `examples/reg_ex1` the output (`q3.out`) is:

```
qf
(S (S "a"))
(S (qf *e*))
```

Q1 notes

- Since only one final state is allowed and for `fsa1` (a^*b^*), a DFA would need two final states ("only a's" and "a's then b's"). So I added an `*e*` arc from `q0` to `q1` and made `q1` the only final state. `fsa1`, `fsa3`, `fsa4` are NFAs with `*e*` arcs and `fsa2` is a DFA.
- For `fsa3` I used a new final state `qf` with `*e*` arcs from the end of each of the two branches, like in Thompson's construction.
- **Checking:** I wrote a small NFA simulator and compared it with Python's `re.fullmatch` on the regular expression, for all strings over $\{a, b, c, d\}$ up to length 7 (about 22,000 strings per FSA). There were no mismatches.

Q2 notes and issues

`fsa_acceptor.py` reads the FSA, checks that it is a DFA, and then runs each input line through it using a dictionary `(state, symbol)` → `next state`.

- **DFA check:** if there is any `*e*` arc, or one `(state, symbol)` pair goes to two different states, it prints `Error: the input FSA is not a DFA` and exits.
- A string is accepted if every symbol has an arc and the last state is the final state. If a symbol has no arc, then the string is rejected. If the start state is the final state, only then, an empty line is accepted.

Issues I ran into:

1. **Quoted input symbols.** At first my program printed "no" for everything on `examples/ex`. The lines in that file have each symbol in quotation marks (like `"a" "a" "b"`), and my code compared `"a"` (with quotes) to `a` (without), so nothing matched. So my fix was that I removed the quotation marks from each input token before the lookup. If a line has no spaces (like `aab`), I split it into characters, so that form works too.
2. **Output format.** The assignment text says `x=> y`, but `examples/ex.output` has a space before the arrow (`x => y`). I followed the example file.
3. **Trailing spaces.** The lines in `examples/ex.output` are not consistent about trailing spaces (some end in `=> no` and some in `=> no`). So I compare with `diff -w` (ignore whitespace), and with that my output matches `examples/ex.output` exactly.

Q4 notes

How I read the three relations:

FST	Relation	Behavior
`fst1`	$a^{2n} \to b^n$	every two a's give one b; odd number of a's is rejected
`fst2`	$a^n \to b^{2n}c$	every a gives bb; one c at the end; empty input gives c
`fst3`	$a^n d^* \to (bc)^n g$	every a gives bc; the d's are deleted; one g at the end

For `fst3` the d's are not counted, because an FST cannot count $a^n d^n$.

I followed the format of `examples/fst0`: symbols are in quotation marks, the final state is on the first line, and the empty string is `*e*` without quotes. `fst2` and `fst3` use `*e*` on the input side (to output the c / g without reading anything), so they are non-deterministic, which is allowed.

I tested them with the simulator, for example:

- `fst1`: $\varepsilon \mapsto \varepsilon$, `aaaa` $\mapsto$ `bb`, `a` rejected
- `fst2`: $\varepsilon \mapsto$ `c`, `a` $\mapsto$ `bbc`, `aa` $\mapsto$ `bbbbc`
- `fst3`: `aadd` $\mapsto$ `bcbcg`, $\varepsilon \mapsto$ `g`, `ddd` $\mapsto$ `g`